

Αξιολόγηση του Cromosim και εφαρμογή στο σύστημα προσομοίωσης Agents.jl

Λειτουργίες και δυνατότητες του Cromosim

Επισκόπηση: Το **Cromosim** είναι ένα ανοικτού κώδικα λογισμικό (βιβλιοθήκη Python) για μικροσκοπική προσομοίωση κίνησης πλήθους ¹. Εστιάζει στην προσομοίωση πεζών σε δισδιάστατο χώρο (2D) και υποστηρίζει πολλαπλά μοντέλα συμπεριφοράς και κινησιολογικές προσεγγίσεις. Συγκεκριμένα, ενσωματώνει:

- **Μοντέλο Social Force (Κοινωνικής Δύναμης):** Αντιμετωπίζει τους πεζούς ως σωματίδια με αδράνεια που κινούνται προς έναν στόχο υπό την επίδραση δυνάμεων ². Οι δυνάμεις αυτές περιλαμβάνουν όρους που αντιπροσωπεύουν την επιθυμία κάθε ατόμου να φτάσει στον προορισμό του καθώς και *απωθητικές δυνάμεις* από άλλους πεζούς και από εμπόδια (τοιχούς), ώστε να διατηρούνται αποστάσεις ασφαλείας ² ³. Το Social Force model έχει γίνει δημοφιλές από τη δεκαετία του 1990 και προσομοιώνει φαινόμενα όπως αποφυγή συγκρούσεων και μποτλιαρίσματα πλήθους μέσω τέτοιων δυναμικών αλληλεπιδράσεων.
- **Μοντέλο “Granular” (Κοκκώδες μοντέλο):** Θεωρεί επίσης κάθε πεζό ως δίσκο (κύκλο) συγκεκριμένης ακτίνας, αλλά αντί να εισάγει ελκτικές/απωθητικές δυνάμεις, επιβάλλει άμεσα την **μη-επικάλυψη των δίσκων** ως περιορισμό ³. Δηλαδή, σε κάθε βήμα χρόνου οι ταχύτητες των ατόμων **προβάλλονται** στον κοντινότερο επιτρεπτό πεδίο ταχυτήτων που δεν οδηγεί σε συγκρούσεις μεταξύ τους ⁴ ⁵. Με πιο απλά λόγια, κάθε άτομο έχει μια επιθυμητή ταχύτητα (σαν να ήταν μόνο του), όμως το σύστημα διορθώνει συλλογικά αυτές τις ταχύτητες ώστε να αποφεύγονται οι επικαλύψεις (συγκρούσεις). Αυτό το μοντέλο βασίζεται σε έννοιες από τη ρευστομηχανική κοκκωδών υλικών και υπολογίζει τις επιτρεπτές κινήσεις μέσω επίλυσης ενός βελτιστοποιητικού προβλήματος ⁵ (το οποίο υλοποιείται με μεθόδους κυρτής βελτιστοποίησης, π.χ. με χρήση της βιβλιοθήκης CVXOPT).
- **Κινησιολογία & Χρονική Εξέλιξη:** Και στα δύο παραπάνω “μικροσκοπικά” μοντέλα, ο χρόνος εξελίσσεται διακριτά (με μικρό βήμα Δt). Οι πράκτορες (πεζοί) έχουν **θέση**, **ταχύτητα** και **ακτίνα σώματος**. Στο Social Force, οι επιταχύνσεις προκύπτουν από τις δυνάμεις και ενημερώνουν τις ταχύτητες/θέσεις κάθε Δt (μέσω αριθμητικής ολοκλήρωσης), ενώ στο Granular μοντέλο αποφεύγονται οι επιταχύνσεις που θα προκαλούσαν επικάλυψη δίσκων, μέσω προβολής των ταχυτήτων στο επιτρεπτό σύνολο ⁴. Ως αποτέλεσμα, και τα δύο μοντέλα προσπαθούν να αποτρέψουν συγκρούσεις: το μεν πρώτο με προσομοίωση *απωθητικών δυνάμεων*, το δε δεύτερο με *γεωμετρικό περιορισμό* (κανένας δύο δίσκοι δεν μπορούν να καταλαμβάνουν τον ίδιο χώρο).
- **Άλλα μοντέλα:** Το Cromosim υποστηρίζει επίσης **απλούστερα μοντέλα**: ένα μονοδιάστατο **Follow-the-Leader** (όπου οι πράκτορες ακολουθούν κατά σειρά, π.χ. αυτοκινητοπομπή ή πεζοί σε σειρά) και ένα **κυτταρικό αυτόματο** 2D για κίνηση πεζών σε διακριτό πλέγμα ⁶. Επιπλέον, περιλαμβάνει ένα *μακροσκοπικό μοντέλο διαμερισμάτων (compartment model)*, όπου ο πληθυσμός προσομοιώνεται σε επίπεδο ρώνων μεταξύ περιοχών αντί για μεμονωμένα άτομα

⁶ . Το compartment model είναι χρήσιμο για εκτιμήσεις πλήθους σε μεγάλης κλίμακας χώρους χωρίς να υπολογίζεται κάθε άτομο ξεχωριστά.

- **Τοξικολογία / Περιβαλλοντικές επιδράσεις:** Στην τρέχουσα έκδοση, το Cromosim **δεν περιλαμβάνει ειδικά μοντέλα τοξικολογίας** ή διάδρασης με χημικούς παράγοντες από προεπιλογή – το επίκεντρο είναι η κίνηση και οι αλληλεπιδράσεις του πλήθους. Ωστόσο, ο σχεδιασμός του είναι επεκτάσιμος: οι πράκτορες μπορούν να έχουν ιδιότητες όπως ένας «συντελεστής ταχύτητας» ή άλλα χαρακτηριστικά που ο χρήστης θα μπορούσε να τροποποιεί. Για παράδειγμα, κάθε άτομο διαθέτει έναν συντελεστή ταχύτητας/αντίδρασης που μπορεί να αρχικοποιηθεί διαφορετικά (π.χ. για γρήγορους ή αργούς πεζούς) ⁷ ⁸ . Θα ήταν θεωρητικά δυνατό ο χρήστης να εισάγει έναν μηχανισμό μείωσης αυτού του συντελεστή όταν ένας πράκτορας βρίσκεται σε τοξικό περιβάλλον, αλλά αυτό δεν είναι έτοιμη λειτουργία της βιβλιοθήκης – θα απαιτούσε πρόσθετο κώδικα από τον χρήστη. Συνοπτικά, η “τοξικολογία” δεν αποτελεί εγγενές μέρος των μοντέλων του Cromosim, αλλά το λογισμικό αφήνει περιθώρια στο χρήστη να προσθέσει τέτοιες επιδράσεις με δική του τροποποίηση.

Περιβάλλον προσομοίωσης και γεωμετρία: Μια από τις ισχυρές δυνατότητες του Cromosim είναι ο τρόπος που ορίζει και διαχειρίζεται το φυσικό περιβάλλον (). Συγκεκριμένα:

- Ο χώρος της προσομοίωσης ορίζεται μέσω μιας εικόνας υποβάθρου (π.χ. ένα σχέδιο κατόψεως, χάρτης χώρου εκδήλωσης ή ακόμα και λευκός καμβάς) πάνω στην οποία ο χρήστης μπορεί να προσθέσει **σχήματα με συγκεκριμένα χρώματα** που αντιστοιχούν σε στοιχεία της σκηνής ⁹ . Για παράδειγμα, **μαύρο χρώμα** μπορεί να σημαίνει **τοίχος/εμπόδιο** (μη προσπελάσιμος χώρος), ενώ μια **κόκκινη γραμμή** μπορεί να σημαίνει **πόρτα/έξοδος** και μια κίτρινη γραμμή μια άλλη πόρτα ⁹ ¹⁰ . Μπορούν να προστεθούν και άλλα σχήματα όπως κύκλοι, ορθογώνια ή πολύγωνα για να διαμορφωθεί η πίστα της προσομοίωσης. Ουσιαστικά, ο χρήστης σχεδιάζει ή φορτώνει την κάτοψη του χώρου και ορίζει έναν **χρωματικό κώδικα** για τα σημαντικά στοιχεία (τοίχοι, έξοδοι, ζώνες κτλ.).
- Με βάση αυτή τη γεωμετρία, το Cromosim **προεπεξεργάζεται αριθμητικά πεδία** που βοηθούν στην πλοήγηση των πρακτόρων. Συγκεκριμένα, παράγεται **πίνακας αποστάσεων από τους πλησιέστερους τοίχους** (για κάθε σημείο του χώρου) ¹¹ και **πίνακες απόστασης προς κάθε πιθανό προορισμό** (π.χ. πόρτα εξόδου) ¹² . Οι πίνακες αυτοί έχουν το ίδιο μέγεθος με την εικόνα (ανάλυση σε pixel) και μπορούν να θεωρηθούν σαν «*χαρτιά δυναμικού*» στον χώρο: κάθε κελί περιέχει την απόσταση μέχρι τον τοίχο ή τον στόχο, αντίστοιχα. Έτσι:
- Το **πεδίο απόστασης από τοίχους** επιτρέπει στον πράκτορα να **αντιλαμβάνεται** πότε πλησιάζει πολύ κοντά σε εμπόδιο. Στο Social Force μοντέλο, αυτή η πληροφορία χρησιμοποιείται για να υπολογιστεί απωθητική δύναμη από τον τοίχο ¹³ . (Η υλοποίηση μάλιστα χρησιμοποιεί τη **μέθοδο fast-marching** για γρήγορο υπολογισμό αποστάσεων στον χώρο με εμπόδια ¹⁴ .)
- Το **πεδίο απόστασης προς τον προορισμό** (π.χ. έξοδο) δίνει για κάθε σημείο πόσο μακριά βρίσκεται από τον στόχο λαμβάνοντας υπόψη τα εμπόδια. Το **αρνητικό_gradient_ αυτού του πεδίου δίνει την κατεύθυνση της συντομότερης πορείας προς τον στόχο** ¹² . Στο Cromosim, αυτή η κατεύθυνση ορίζεται ως η **επιθυμητή κατεύθυνση/ταχύτητα** κάθε πράκτορα. Εν ολίγοις, οι πράκτορες “ξέρουν” πού να κινηθούν για να βγουν από τον χώρο ακολουθώντας την ευκολότερη διαδρομή γύρω από εμπόδια, χωρίς να χρειάζεται ο χρήστης να προγραμματίσει αλγορίθμους διαδρομής – το σύστημα δημιουργεί αυτόματα ένα **πεδίο καθοδήγησης** προς την έξοδο.

Η απόσταση προς μια έξοδο (χρωματική απεικόνιση) και η κατεύθυνση της επιθυμητής ταχύτητας (βέλη) όπως υπολογίζονται από το *Cromosim*. Οι πράκτορες χρησιμοποιούν αυτά τα πεδία για να κινούνται προς τον στόχο αποφεύγοντας εμπόδια (μαύρη περιοχή).

- Το *Cromosim* επιτρέπει ορισμό πολλαπλών **προορισμών (Destinations)**. Κάθε *Destination* έχει ένα χρώμα στον χάρτη (π.χ. κόκκινο για την κύρια έξοδο, κίτρινο για δευτερεύουσα) και συνοδεύεται από τον δικό του πίνακα αποστάσεων/κατευθύνσεων ¹⁰. Μάλιστα, μπορεί να οριστεί **αλυσίδα προορισμών**: π.χ. πρώτα μια κόκκινη γραμμή που συμβολίζει την πόρτα μιας σκάλας, μετά η ίδια η σκάλα ως προορισμός που οδηγεί σε άλλο επίπεδο (άλλο Domain) ¹⁵. Με αυτόν τον τρόπο υποστηρίζεται η προσομοίωση **πολλαπλών επιπέδων/ορόφων**: μπορούμε να έχουμε ένα αντικείμενο Domain ανά όροφο, με αντίστοιχες εξόδους (π.χ. σκάλα ανόδου/καθόδου), και να μεταφέρονται οι πράκτορες από τον ένα όροφο στον άλλο όταν φτάνουν στη σκάλα. Οι σκάλες ορίζονται πάλι ως χρωματιστές γραμμές (π.χ. μπλε) και συνδέονται με προορισμούς σε άλλον Domain (όροφο) ¹⁵. Έτσι, το *Cromosim* μπορεί να προσομοιώσει π.χ. μια εκκένωση σε κτίριο με ορόφους και σκάλες.

- **Εμπόδια και σύγκρουση με τοίχους**: Οι τοίχοι ή άλλα εμπόδια ορίζονται (συνήθως σε μαύρο χρώμα). Οι πράκτορες δεν μπορούν να κινηθούν μέσα από τις μαύρες περιοχές. Το λογισμικό χρησιμοποιεί τις πληροφορίες απόστασης από τοίχο για να αποτρέψει την κίνηση εκεί: στο *Social Force* προσθέτει δύναμη ώθησης όταν η απόσταση `d` γίνεται μικρή (κάτω από κάποιο όριο) ¹⁴, ενώ στο *Granular* μοντέλο πρακτικά δεν θα υπάρχει επιτρεπτό πεδίο ταχύτητας που να οδηγεί κάποιον μέσα σε τοίχο (οι τοίχοι θεωρούνται *εξαιρούμενες ζώνες* στον υπολογισμό του πεδίου προορισμού) ¹⁰.

- **Πληθυσμός και αρχικοποίηση**: Οι πράκτορες (άνθρωποι) μπορούν να χωριστούν σε ομάδες κατά την αρχικοποίηση. Το *Cromosim* δίνει τη δυνατότητα να οριστεί **τυχαία κατανομή** για κάποιες ιδιότητες τους – π.χ. η ακτίνα σώματος και η μέγιστη ταχύτητά τους να δειγματολογούνται από **κανονική ή ομοιόμορφη κατανομή** αντί να είναι όλα ίδια ¹⁶. Έτσι προσομοιώνεται η φυσική ετερογένεια σε μέγεθος και ταχύτητα βάδισης του πληθυσμού. Επιπλέον, σε κάθε ομάδα μπορεί να αποδοθεί εξαρχής διαφορετικός προορισμός (π.χ. κάποιοι άνθρωποι να κατευθύνονται στην έξοδο Α και άλλοι στην έξοδο Β από την αρχή) ¹⁷.

Απεικόνιση και ανάλυση αποτελεσμάτων: Το *Cromosim* διαθέτει ενσωματωμένες λειτουργίες για την **απεικόνιση** της προσομοίωσης και την **καταγραφή μετρήσεων**:

- Παρέχονται συναρτήσεις για να σχεδιάζονται οι πράκτορες και οι τροχιές τους σε διδιάστατα γραφήματα. Συγκεκριμένα, η συνάρτηση `plot_people()` σχεδιάζει **κύκλους** για κάθε άτομο (με ακτίνα ανάλογη του μεγέθους τους) πάνω στο σχήμα του χώρου, **βέλη** για τις ταχύτητές τους, και γραμμές που ενώνουν όσους βρίσκονται σε επαφή (συγκρούσεις) εκείνη τη στιγμή ⁷ ¹⁸. Ο χρωματισμός των πρακτόρων μπορεί να απεικονίζει κάποια ποσότητα (π.χ. την απόσταση από την έξοδο ή άλλη μεταβλητή) μέσω χρωματικής κλίμακας ¹⁹ ²⁰. Υπάρχει επιλογή να εμφανίζονται οι διαδρομές που έχουν διανύσει (ίχνη) καθώς και οι “αισθητήρες” (δες επόμενη παράγραφο) ¹⁸. Οι εικόνες μπορούν να αποθηκευτούν σε αρχείο (png) για δημιουργία βίντεο ή ανάλυση ²¹. Η απόδοση αυτών γίνεται μέσω της *Matplotlib*, οπότε το αποτέλεσμα είναι στατικά γραφήματα 2D.
- Εκτός από την οπτική απεικόνιση της κίνησης, το *Cromosim* ενσωματώνει ένα μηχανισμό **“αισθητήρων” (pedestrian sensors)** για τη συλλογή δεδομένων. Ουσιαστικά, ένας αισθητήρας ορίζεται ως μια εικονική γραμμή σε μια έξοδο ή διάδρομο, και το λογισμικό μπορεί να μετράει **πόσα άτομα διέρχονται** από αυτή τη γραμμή και σε ποιο χρόνο ²² ²³. Για κάθε αισθητήρα/

γραμμή διέλευσης, η Cromosim παρέχει στατιστικά όπως: αριθμός εισερχομένων και εξερχομένων ατόμων, χρονικές στιγμές περάσματος, καθώς και ροές (πλήθος ατόμων ανά μονάδα χρόνου) ²⁴ ²⁵. Επιπλέον, προσφέρει συνάρτηση `plot_sensors()` που παράγει αυτόματα **γραφήματα ροής-χρόνου** για κάθε αισθητήρα ²⁶ ²⁷. Για παράδειγμα, στο παρεχόμενο παράδειγμα εκκένωσης, δύο αισθητήρες τοποθετήθηκαν σε δύο εξόδους και το Cromosim παράγει γραφήματα που δείχνουν τον αριθμό ατόμων που πέρασαν από κάθε έξοδο στο χρόνο, καθώς και τη ροή (άτομα ανά δευτερόλεπτο) ²⁸. Αυτά τα εργαλεία διευκολύνουν τον αναλυτή να μελετήσει αποτελέσματα όπως χρόνους εκκένωσης, συμφόρηση σε πόρτες κλπ., χωρίς πρόσθετη επεξεργασία.

Συνολικά, το Cromosim παρέχει μια **ολοκληρωμένη εργαλειοθήκη για προσομοίωση πλήθους**: από τον ορισμό σύνθετης γεωμετρίας και πολλαπλών εξόδων, μέχρι τη λεπτομερή δυναμική αλληλεπίδραση των πρακτόρων και την εξαγωγή χρήσιμων μετρήσεων. Αποτελεί μια **εξιδανικευμένη υλοποίηση** πολλών μοντέλων που περιγράφονται στη βιβλιογραφία της προσομοίωσης πλήθους ²⁹, με στόχο την ευκολία χρήσης: επιτρέπει στον χρήστη να τρέχει έτοιμα μοντέλα ή να δοκιμάζει νέες ιδέες σε ένα συνεκτικό πλαίσιο.

Τεχνολογική υποδομή και τρόπος λειτουργίας του Cromosim

Το Cromosim είναι υλοποιημένο ως **βιβλιοθήκη Python**. Η επίσημη έκδοση (Cromosim 2.x) μπορεί να εγκατασταθεί μέσω `pip` ³⁰ και ο κώδικας του είναι διαθέσιμος στο GitHub (άδεια GPL). Η διάρθρωση του λογισμικού έχει ως εξής:

- **Τρόπος Χρήσης:** Δεν διαθέτει αυτόνομο γραφικό περιβάλλον (GUI), αλλά χρησιμοποιείται μέσω **εκτέλεσης Python scripts**. Ο χρήστης είτε γράφει το δικό του script ή τροποποιεί τα παρεχόμενα παραδείγματα. Για παράδειγμα, το αποθετήριο περιλαμβάνει έτοιμα scripts (π.χ. `micro_social.py`, `micro_granular.py`) τα οποία μπορούν να τρέξουν με ορισμένες παραμέτρους εισόδου (π.χ. `--json input_room.json`) ³¹ ³². Αυτά τα αρχεία JSON περιγράφουν το σενάριο: π.χ. όνομα και μέγεθος του χώρου, αρχείο εικόνας υποβάθρου, λίστες συντεταγμένων για γραμμές τοίχων και θυρών, καθώς και άλλες ρυθμίσεις όπως αριθμό πρακτόρων, ομάδες κ.ά. ³³ ³⁴. Έτσι, ο χρήστης μπορεί να ορίσει μια σκηνή είτε γράφοντας JSON (ή Python κώδικα) αντί να σχεδιάζει μέσω GUI. Υπάρχουν παραδείγματα JSON αρχείων εισόδου που συνοδεύουν τα scripts, ώστε να τροποποιούνται για νέα σενάρια.
- **API και Δομή Κώδικα:** Το Cromosim οργανώνει τις λειτουργίες του σε διάφορα modules. Υπάρχει το module `cromosim.domain` (για κατασκευή χώρου/Domain και Destination), το `cromosim.micro` (που περιλαμβάνει τις συναρτήσεις για τα μικροσκοπικά μοντέλα Social/Granular: υπολογισμός επαφών, δυνάμεων, μετακίνηση πρακτόρων κλπ.), modules για το compartment model και το follow-the-leader, κ.ά. ³⁵ ³⁶. Ο πυρήνας προσομοίωσης υλοποιείται με έναν βρόχο επαναλήψεων στον χρόνο: κάθε βήμα καλεί τις κατάλληλες συναρτήσεις (ανάλογα το μοντέλο) για:
- **Εύρεση γειτόνων/συγκρούσεων:** Χρησιμοποιείται αποδοτική δομή (δέντρο k-d) για να βρεθούν τα ζεύγη πρακτόρων που είναι κοντά ³⁷. Επίσης, για κάθε πράκτορα ελέγχεται η απόσταση από τους πλησιέστερους τοίχους (με χρήση του προϋπολογισμένου πεδίου αποστάσεων ή μεθόδου fast-marching) ¹⁴. Αυτές οι πληροφορίες τροφοδοτούν είτε τον υπολογισμό δυνάμεων (Social Force) είτε τον αλγόριθμο projection (Granular).
- **Υπολογισμό νέων ταχυτήτων:** Στο Social Force, για κάθε πράκτορα υπολογίζεται η συνισταμένη δύναμη (από την επιθυμία προς τον στόχο + απωθητικές από γείτονες + απωθητικές από τοίχους) ³⁸ ³⁹. Αυτή μετασχηματίζεται (με $F = m a$) σε επιτάχυνση και

ενημερώνει την ταχύτητα. Στο Granular μοντέλο, καλείται η ρουτίνα `projection()` που λαμβάνει όλες τις επιθυμητές ταχύτητες και διορθώνει συλλογικά τις τιμές τους επιλύοντας το πρόβλημα μη-επικάλυψης (το οποίο υλοποιείται αριθμητικά, π.χ. με solver) ⁴⁰.

- **Μετακίνηση πρακτόρων:** Οι νέες ταχύτητες εφαρμόζονται για να ενημερωθούν οι θέσεις (π.χ. $x_{\text{new}} = x + v \Delta t$), μέσω συνάρτησης `move_people()` ⁴¹. Επιπλέον, γίνονται έλεγχοι ώστε κανείς να μην βρεθεί εκτός ορίων του χώρου (ή μέσα σε τοίχο – τυχόν αστοχίες διορθώνονται μετακινώντας τον πράκτορα λίγο πίσω σε έγκυρη θέση) ⁴² ⁴³. Αν ένας πράκτορας φτάσει σε μια γραμμή προορισμού (π.χ. πόρτα εξόδου), τότε είτε τερματίζεται (αν ήταν τελική έξοδος) είτε μεταφέρεται στον επόμενο προορισμό/όροφο (π.χ. αρχίζει να κινείται στο Domain του επόμενου ορόφου ως “νέος” πράκτορας εκεί).
- **Καταγραφή δεδομένων/οπτικοποίηση:** Αν ο χρήστης έχει ενεργοποιήσει την καταγραφή αισθητήρων, σε κάθε βήμα ελέγχεται ποιοι πράκτορες διέσχισαν τις καθορισμένες γραμμές από το προηγούμενο βήμα ⁴⁴ ²⁴. Τα συμβάντα (είσοδος/έξοδος ανά αισθητήρα) αποθηκεύονται σε δομές για μεταγενέστερη ανάλυση. Παράλληλα, μπορεί να γίνεται ενημέρωση γραφικών σε πραγματικό χρόνο – π.χ. το script παραδείγματος καλεί `plt.pause()` μέσα στο βρόχο για να ανανεώνει την εικόνα με τις θέσεις των πρακτόρων ⁴⁵, επιτρέποντας μια μορφή *κινούμενης visualization* (αν και όχι διαδραστικής πέρα από παύση/συνέχιση).
- **Απόδοση και κλίμακα:** Το Cromosim, όντας υλοποιημένο σε Python, κάνει εκτεταμένη χρήση βιβλιοθηκών NumPy/SciPy για να επιταχύνει τους βαρείς υπολογισμούς. Για παράδειγμα, οι πράκτορες και οι ιδιότητές τους αποθηκεύονται σε numpy arrays, ώστε οι υπολογισμοί μεταξύ όλων των ζευγών (ή η επίλυση του projection) να γίνονται σε βελτιστοποιημένο κώδικα (C/C++ routines μέσω numpy). Η χρήση της δομής kd-Tree από τη SciPy για εύρεση γειτόνων επίσης βελτιώνει την πολυπλοκότητα (από $O(N^2)$ brute force σε περίπου $O(N \log N)$ κατά μέσο όρο) ¹⁴. Παρ’ όλα αυτά, επειδή μεγάλο μέρος της λογικής είναι γραμμένο σε Python (ιδίως ο βρόχος προσομοίωσης, έλεγχοι, κλπ.), η απόδοση για **πολύ μεγάλα πλήθη** (π.χ. εκατοντάδων χιλιάδων πρακτόρων) θα ήταν περιορισμένη από τον GIL και την ταχύτητα της Python. Σε τυπικά μεγέθη (εκατοντάδες ή λίγες χιλιάδες πράκτορες) θεωρείται επαρκές, με το βαρύτερο κομμάτι (αλγόριθμοι υπολογισμού αποστάσεων/επιλύτες) να εκτελείται σε native κώδικα. Δεν γίνεται εγγενής παράλληλη εκτέλεση σε πολλούς πυρήνες από τη μεριά του Cromosim (η Python εκτέλεση είναι κύρια σειριακή, αν και η NumPy/BLAS μπορεί εσωτερικά να αξιοποιήσει πολλαπλούς πυρήνες για συγκεκριμένες πράξεις).
- **Διασυνδεσιμότητα και πρότυπα:** Το Cromosim δεν εισάγει κάποιο ιδιόκτητο format – στηρίζεται σε **ανοιχτά format** (PNG/JPEG εικόνες για γεωμετρία, JSON για παραμέτρους). Δεν παρέχει ειδική υποστήριξη **GIS** (π.χ. δεν διαβάζει shapefiles ή δεδομένα χάρτη απευθείας). Ωστόσο, επειδή μπορεί να δουλέψει με εικόνες ως χάρτες υποβάθρου, θα μπορούσε κανείς να χρησιμοποιήσει ένα σχέδιο χώρου από GIS (π.χ. εξαγόμενο ως εικόνα) ή δεδομένα OpenStreetMap μετασχηματισμένα σε εικόνα/JSON εισόδου. Επίσης, το Cromosim ακολουθεί απλές δομές που θα ήταν συμβατές με άλλα εργαλεία: π.χ. η λίστα συντεταγμένων για τοίχους/πόρτες από το JSON θα μπορούσε να εξαχθεί ή να εισαχθεί από άλλο λογισμικό σχετικά εύκολα. Δεν υπάρχει όμως απευθείας ολοκλήρωση με OSM ή άλλα πρότυπα – ο χρήστης πρέπει να μετατρέψει τα δεδομένα του στο format που περιμένει το Cromosim.

Συνοψίζοντας, τεχνολογικά το Cromosim είναι μια **βιβλιοθήκη Python χωρίς GUI**, που αξιοποιεί αλγορίθμους αριθμητικής ανάλυσης (π.χ. fast marching, kd-tree, convex optimization) για να προσφέρει αξιόπιστη προσομοίωση. Η έμφαση του είναι στην **ευκολία**: καλά τεκμηριωμένα παραδείγματα, απλή παράμετροποίηση μέσω αρχείων, και δυνατότητα να δοκιμάσει κανείς εναλλακτικά μοντέλα τροποποιώντας/επεκτείνοντας τον κώδικα (do-it-yourself προσέγγιση ²⁹). Δεν

προσφέρει φιλικό γραφικό περιβάλλον, ούτε τρισδιάστατη απεικόνιση – λειτουργεί περισσότερο ως **εργαλείο προγραμματιστή/ερευνητή** για πειραματισμό με σενάρια πλήθους.

Σύγκριση Cromosim με Agents.jl και το υπάρχον μοντέλο

Εφόσον το υπάρχον σύστημα προσομοίωσης έχει υλοποιηθεί σε Julia με χρήση του **Agents.jl**, είναι κρίσιμο να συγκριθούν τα δύο περιβάλλοντα ως προς τις δυνατότητές τους και το κατά πόσο οι λειτουργίες του Cromosim μπορούν να αντικατοπτριστούν στο Agents.jl. Στον παρακάτω πίνακα παρουσιάζεται μια σύνοψη της σύγκρισης:

Χαρακτηριστικό	Cromosim (Python)	Agents.jl (Julia)
Εστίαση/ Προορισμός	Προσομοίωση κίνησης πλήθους (πεζών) σε 2D ⁴⁶ . Εξειδικευμένα μοντέλα και εργαλεία για πλήθη.	Γενικό πλαίσιο ABM (Agent-Based Modeling) για κάθε είδους πράκτορες. Δεν είναι ειδικό για πλήθη, αλλά μπορεί να χρησιμοποιηθεί για αυτά.
Διάσταση χώρου	2D συνεχής χώρος (επίπεδο). Δεν υποστηρίζει 3D κίνηση εκτός αν θεωρηθούν πολλοί 2D όροφοι.	Υποστηρίζει 2D και 3D <i>ContinuousSpace</i> για πράκτορες ⁴⁷ ⁴⁸ . Μπορεί να τοποθετήσει πράκτορες και σε γράφους ή πλέγματα.
Γεωμετρία περιβάλλοντος	Ορίζεται από εικόνες/σχήματα. Εσωτερική αναπαράσταση μέσω πινάκων αποστάσεων σε πλέγμα (pixel grid) ¹¹ ¹² . Ενσωματωμένη λογική για τοίχους, πόρτες, ορόφους.	Δεν υπάρχει προκαθορισμένη αναπαράσταση γεωμετρίας. Ο χρήστης ορίζει εμπόδια/όρια μέσω κώδικα (π.χ. λίστες τοίχων, ή μάσκα πλέγματος). Υπάρχει όμως ενσωμάτωση OpenStreetMap για χρήση πραγματικών οδικών δικτύων ως χώρου ⁴⁹ .
Μοντέλα κίνησης/ συμπεριφοράς	Social Force, Granular (συνεχής χώρος), Cellular Automata, Follow-leader, Compartment (μακροσκοπικό) ⁶ . Παρέχονται έτοιμα.	Δεν παρέχει έτοιμα μοντέλα πλήθους. Ο χρήστης υλοποιεί τους κανόνες συμπεριφοράς των πρακτόρων (π.χ. μπορεί να κωδικοποιήσει social force ή άλλο μοντέλο χειρωνακτικά). Υπάρχουν παραδείγματα (π.χ. flocking/boids) που μπορούν να αξιοποιηθούν ως οδηγός.

Χαρακτηριστικό	Cromosim (Python)	Agents.jl (Julia)
Κινητική ενημέρωση	Χρονικά βήματα με υπολογισμό δυνάμεων/επαφών και ενημέρωση θέσεων αυτόματα από τη βιβλιοθήκη ³ ¹⁴ . Συμπεριλαμβάνει αποφυγή συγκρούσεων (με δυνάμεις ή projection) εγγενώς.	Χρονικά βήματα με user-defined step function. Το Agents.jl προσφέρει βοηθητικές δομές (π.χ. spatial index για εύρεση γειτόνων) αλλά ο χρήστης είναι υπεύθυνος να ενημερώσει θέσεις/ταχύτητες, να ελέγξει συγκρούσεις κλπ. Δεν υπάρχει αυτόματη αποφυγή συγκρούσεων εκτός αν την κωδικοποιήσει ο χρήστης.
Ιδιότητες πρακτόρων	Στατικές και δυναμικές ιδιότητες προκαθορισμένες: θέση (x,y), ταχύτητα, ακτίνα, συντελεστής ταχύτητας ν, προορισμός, κ.ά. ⁸ . Ο χρήστης μπορεί να προσθέσει επιπλέον αλλά τροποποιώντας κώδικα.	Ο χρήστης ορίζει τυχαίου τύπου ιδιότητες στη δομή του agent (π.χ. toxic_load, health, speed_factor κ.ο.κ.). Πολύ ευέλικτο σχήμα – με το μακρο @agent μπορούν εύκολα να προστεθούν πεδία όπως “τοξικό φορτίο”.
Επίδραση περιβάλλοντος (π.χ. τοξικότητα)	Δεν υπάρχει έτοιμο μοντέλο για περιβαλλοντικούς παράγοντες. Θα απαιτούσε πρόσθετη λογική ενσωματωμένη στο βρόχο (δεν υπάρχει API plugin, θα έπρεπε ο χρήστης να τροποποιήσει τον κώδικα των μοντέλων για να προσθέσει π.χ. μείωση ταχύτητας λόγω τοξικού αερίου).	Αφήνεται πλήρως στον χρήστη. Το υπάρχον μοντέλο σας π.χ. ήδη υλοποιεί συγκέντρωση τοξικού σε πλέγμα και επίδραση στην ταχύτητα των πρακτόρων – αυτό είναι εφικτό επειδή ο κώδικας εξελικτικών κανόνων είναι ανοιχτός. Το Agents.jl δεν περιορίζει τέτοιες προσθήκες.
Οπτικοποίηση	2D plots μέσω Matplotlib. Στατική απεικόνιση ή βασική animation εντός notebook. Υποστηρίζει σχεδίαση πρακτόρων/τροχιών, εντάσεων ροής, κλπ. ⁷ ¹⁸ . Δεν διαθέτει διαδραστικό 3D ή GUI προβολέα.	Διάφορες επιλογές: π.χ. χρήση AgentsPlots.jl (βασισμένο σε Makie ή άλλες βιβλιοθήκες) για 2D/3D γραφική αναπαράσταση. Μπορεί να γίνει και live animation σε ένα GLFW παράθυρο ή σε Pluto notebook. Όχι ενιαίο έτοιμο GUI, αλλά μπορεί να δημιουργηθεί π.χ. με Makie sliders για real-time έλεγχο.
Συλλογή δεδομένων	Ενσωματωμένοι αισθητήρες πεζών (μέτρηση διελεύσεων, χρόνων, ροών) και αυτόματη δημιουργία γραφημάτων ²⁴ ²⁶ . Δυνατότητα αποθήκευσης αποτελεσμάτων (png, JSON logs).	Agents.jl διαθέτει datacollection μηχανισμό: μπορεί εύκολα να συλλέγει αυθαίρετα μεγέθη ανά βήμα (π.χ. μέση ταχύτητα, αριθμός πρακτόρων σε περιοχή κ.ά.) σε πίνακες/DataFrames. Επίσης υποστηρίζει batch runs και ανάλυση πολλαπλών παραμέτρων. Δεν έχει προκαθορισμένα “αισθητήρια”, αλλά ο χρήστης μπορεί να γράψει λογική για μέτρηση διελεύσεων όπως στο Cromosim.

Χαρακτηριστικό	Cromosim (Python)	Agents.jl (Julia)
Επιδόσεις & κλιμάκωση	Python+NumPy, επαρκής για μεσαίου μεγέθους πλήθη. Μπορεί να επιβραδυνθεί με πολλούς πράκτορες λόγω GIL. Δεν υποστηρίζει παράλληλη εκτέλεση σε πολλαπλούς πυρήνες (εκτός ίσως από ενσωματωμένες συναρτήσεις numpy).	Η Julia συνήθως προσφέρει υψηλότερη ταχύτητα σε καθαρό υπολογισμό. Το Agents.jl έχει σχεδιαστεί με απόδοση κατά νου ⁵⁰ . Υποστηρίζει πολυνηματική ή και κατανεμημένη προσομοίωση (ο χρήστης μπορεί να τρέξει παράλληλα πολλά σενάρια ή να εκμεταλλευτεί threads για ενημέρωση πρακτόρων) ⁴⁹ . Για μεγάλο N, η Julia θα scales καλύτερα.
Ευχρηστία & κώδικας	Πολύ εξειδικευμένο – λίγες γραμμές Python φτάνουν για να τρέξει ένα σενάριο με ρεαλιστική συμπεριφορά, αφού η βιβλιοθήκη κρύβει την πολυπλοκότητα ⁵¹ . Αλλά η παραμετροποίηση πέραν των διαθέσιμων επιλογών απαιτεί κατανόηση/τροποποίηση του κώδικα της βιβλιοθήκης (όχι πολύ σχολιασμένου σύμφωνα με κριτικές ⁵²).	Γενικού σκοπού – χρειάζονται περισσότερες γραμμές κώδικα για να φτιάξει κανείς ένα πλήρες μοντέλο πλήθους (διότι πρέπει να γραφούν οι κανόνες). Ωστόσο, ο κώδικας παραμένει σχετικά απλός/σύντομος λόγω της εκφραστικότητας της Julia και της σχεδίασης του Agents.jl ⁵⁰ . Με λίγη προσπάθεια μπορεί να προκύψει μοντέλο προσαρμοσμένο ακριβώς στις ανάγκες του χρήστη.
Συντήρηση & κοινότητα	Συντηρείται από ακαδημαϊκούς (Paris-Saclay). Νέα έκδοση (2.1.0) το 2024, ενεργό project αλλά μικρής κοινότητας (λίγα αστέρια στο GitHub). Τεκμηρίωση πλήρης με θεωρία και παραδείγματα ⁵³ .	Το Agents.jl αναπτύσσεται στο πλαίσιο του JuliaDynamics και έχει μια ενεργή κοινότητα χρηστών της Julia ABM. Υπάρχει επιστημονική δημοσίευση γι' αυτό ⁵⁰ και αρκετά παραδείγματα/οδηγοί. Η ευρύτερη κοινότητα Julia ενισχύει τη συντήρηση.

Συζήτηση σύγκρισης: Από τον πίνακα, είναι εμφανές ότι το Cromosim και το Agents.jl ακολουθούν διαφορετικές φιλοσοφίες:

- Το Cromosim είναι μια **ειδικευμένη πλατφόρμα πλήθους**: παρέχει έτοιμες υλοποιήσεις καθιερωμένων μοντέλων (social force, granular κ.ά.) και αυτοματοποιεί πολλές λεπτομέρειες (αποστάσεις, αποφυγή συγκρούσεων, πλοήγηση). Αυτό σημαίνει ότι ένας χρήστης μπορεί πολύ **γρήγορα** να τρέξει μια προσομοίωση εκκένωσης ή κυκλοφορίας πεζών χωρίς να υλοποιήσει ο ίδιος τις υποκείμενες εξισώσεις. Η μάθηση και χρήση του Cromosim διευκολύνεται από την τεκμηρίωση και τα παραδείγματα του (μικρά σενάρια με λίγες γραμμές κώδικα) ⁵². Ωστόσο, αυτή η ευκολία συνοδεύεται από **λιγότερη ευελιξία** εκτός του πεδίου για το οποίο φτιάχτηκε: αν ο χρήστης θελήσει να προσθέσει ένα φαινόμενο πέρα από αυτά που προβλέπει η βιβλιοθήκη (π.χ. τοξική έκθεση, ή ένα διαφορετικό πρότυπο συμπεριφοράς), θα χρειαστεί να τροποποιήσει τον κώδικα του Cromosim σε χαμηλό επίπεδο. Αυτό μπορεί να είναι δύσκολο, καθώς ο κώδικας «δεν είναι πολύ σχολιασμένος και φαίνεται ακατάστατος» σύμφωνα με αξιολογήσεις ⁵². Εν ολίγοις, το Cromosim παρέχει **έτοιμες λύσεις με περιορισμένη επεκτασιμότητα** από μη ειδικούς.

- Το Agents.jl, αντιθέτως, είναι ένα **γενικό πλαίσιο ABM**. Δεν γνωρίζει εκ των προτέρων τι σημαίνει “πράκτορας πλήθους” ή “τοίχος” – αυτά όλα είναι έννοιες που εισάγει ο χρήστης. Η βιβλιοθήκη φροντίζει για τη **δομή της προσομοίωσης** (π.χ. την αποθήκευση των πρακτόρων, την αποδοτική εύρεση γειτόνων, τη χρονική οργάνωση, συλλογή δεδομένων) ⁵⁴, αλλά οι **κανόνες εξέλιξης** πρέπει να προγραμματιστούν. Αυτό απαιτεί περισσότερο χρόνο αρχικά, όμως δίνει **απόλυτη ελευθερία**: το υπάρχον μοντέλο σας με τοξικότητα, για παράδειγμα, υλοποιήθηκε ακριβώς επειδή το Agents.jl επιτρέπει αυθαίρετους κανόνες – ο χρήστης μπορεί να ορίσει ότι η ταχύτητα του agent μειώνεται ανάλογα με το `toxic_load` ή ότι πεθαίνει αν ξεπεράσει ένα όριο, χωρίς να προσκρούει σε περιορισμούς πλατφόρμας. Επιπλέον, το Agents.jl ενσωματώνεται καλά με το υπόλοιπο οικοσύστημα Julia: μπορούμε να εκμεταλλευτούμε βιβλιοθήκες για επίλυση διαφορικών εξισώσεων, βελτιστοποίησης, επεξεργασίας εικόνας, κ.ά., κατευθειάν μέσα στο ABM. Για παράδειγμα, αν χρειαζόμαστε έναν αλγόριθμο fast-marching ή A*, υπάρχει η δυνατότητα να χρησιμοποιήσουμε υπάρχουσες υλοποιήσεις Julia δίπλα στον κώδικα πρακτόρων.
- **Στο θέμα της κίνησης σε συνεχή χώρο με εμπόδια**: Το Cromosim προσφέρει από μόνο του μια **ολοκληρωμένη λύση πλοήγησης** (distance fields & gradient-following). Στο Agents.jl, η αντίστοιχη λειτουργικότητα θα πρέπει να υλοποιηθεί από τον χρήστη (βλ. επόμενη ενότητα για το πώς). Ωστόσο, αξίζει να σημειωθεί ότι το Agents.jl **υποστηρίζει έμμεσα GIS δεδομένα** μέσω του `OpenStreetMapSpace` ⁴⁹. Αυτό σημαίνει ότι αν θέλαμε να προσομοιώσουμε πλήθος σε μια αστική περιοχή, θα μπορούσαμε να εισάγουμε τον πραγματικό οδικό χάρτη ως γράφο κίνησης. Το Cromosim, αντίθετα, θα απαιτούσε να φτιάξουμε μια εικόνα της περιοχής και να κωδικοποιήσουμε δρόμους ως βατές περιοχές και κτίρια ως εμπόδια χειροκίνητα. Επομένως, για **σενάρια μεγάλης κλίμακας με γεωγραφικά δεδομένα**, το Agents.jl μπορεί να προσφέρει συντομεύσεις (χάρη στην Julia και τις βιβλιοθήκες της), ενώ το Cromosim θα ήθελε πιο χειρωνακτική προετοιμασία.
- **Όσον αφορά το υπάρχον μοντέλο με τοξικότητα**: Το Cromosim εκ πρώτης όψεως δεν μπορεί να το αναπαράγει άμεσα, διότι δεν καταλαβαίνει τι είναι «τοξικός παράγοντας». Θα έπρεπε να εισαχθεί ως ένα επιπλέον πεδίο στο Domain (π.χ. ένας χάρτης συγκέντρωσης) και ως ιδιότητα στους πράκτορες (σωρευμένη δόση). Ενώ αυτό δεν είναι αδύνατο (λόγω ανοιχτού κώδικα), **δεν υπάρχει μηχανισμός** στο Cromosim που να λέει “μείωσε την ταχύτητα αν η συγκέντρωση στο σημείο σου είναι X” – θα έπρεπε ο χρήστης να τροποποιήσει τις συναρτήσεις κίνησης. Αντίθετα, στο Agents.jl αυτό υλοποιείται φυσικά: στο βήμα ενημέρωσης του κάθε agent, ο κώδικας μπορεί να διαβάσει την τιμή τοξικότητας από έναν πίνακα στο περιβάλλον και να ενημερώσει την ιδιότητα ταχύτητάς του ή να τον αφαιρέσει αν πεθαίνει. Εν ολίγοις, **το Agents.jl υπερέχει στην ένταξη προσαρμοσμένων φαινομένων** (π.χ. τοξικότητα, κόπωση, πανικός) επειδή αυτά ενσωματώνονται εύκολα ως κανόνες, ενώ το Cromosim υπερέχει στο ότι **παρέχει εξ αρχής ρεαλιστική κίνηση πλήθους** χωρίς ο χρήστης να χρειάζεται να υλοποιήσει φυσικές δυνάμεις και αλγόριθμους αποφυγής.
- **Visualization & διεπαφή**: Και οι δύο λύσεις υστερούν σε φιλικότητα προς μη προγραμματιστές. Το Cromosim δεν έχει GUI κατ’ ουσίαν – όλα γίνονται μέσω κώδικα/εντολών, ενώ και η απεικόνιση είναι offline γραφήματα. Το Agents.jl από μόνο του επίσης δεν προσφέρει GUI, αλλά επειδή είναι μέσα στη Julia μπορεί κάποιος να φτιάξει, π.χ., ένα **Pluto notebook** με διαδραστικά στοιχεία για να ρυθμίζει παραμέτρους και να δει αποτελέσματα σε πραγματικό χρόνο, ή μια μικρή εφαρμογή με **Makie.jl** που να δείχνει κινούμενες κουκκίδες για τους πράκτορες. Αυτά όμως απαιτούν έξτρα ανάπτυξη. Στον πυρήνα και τα δύο είναι **πλατφόρμες για ανάπτυξη μοντέλων**, όχι έτοιμα εργαλεία με κουμπιά και sliders όπως ίσως άλλες εμπορικές λύσεις (AnyLogic, Pathfinder κλπ.).

Συμπερασματικά, το Agents.jl σε σχέση με το Cromosim προσφέρει **μεγαλύτερη ευελιξία και επεκτασιμότητα**, πληρώνοντας όμως το τίμημα της **μη παρεμβατικότητας** – ο χρήστης πρέπει να ορίσει μόνος του τους κανόνες κίνησης και αλληλεπίδρασης που στο Cromosim είναι ήδη υλοποιημένοι. Από την άλλη, ό,τι κάνει το Cromosim θεωρητικά μπορεί να γίνει και στο Agents.jl (λόγω της πληρότητας του Julia ως γλώσσας και της απόδοσης της), απλώς χρειάζεται σχεδιασμό και προσπάθεια από την πλευρά του προγραμματιστή.

Αναπαραγωγή δυνατοτήτων του Cromosim στο οικοσύστημα Julia/Agents.jl

Δεδομένης της παραπάνω σύγκρισης, θα εξετάσουμε συγκεκριμένα **πώς μπορούν να υλοποιηθούν στο Agents.jl οι βασικές λειτουργίες του Cromosim**. Οι προτάσεις αυτές αποσκοπούν στο να φέρουν το υπάρχον μοντέλο (Agents.jl + τοξικότητα) πιο κοντά στις δυνατότητες μιας πλήρους πλατφόρμας crowd simulation, αξιοποιώντας βιβλιοθήκες και τεχνικές στη Julia:

- **Μοντέλο Social Force (Αποφυγή συγκρούσεων με δυνάμεις):** Μπορεί να υλοποιηθεί στο Agents.jl με τη δημιουργία μιας συνάρτησης που, για κάθε πράκτορα, υπολογίζει τις δυνάμεις από τους γείτονές του και από τα εμπόδια:
- Το Agents.jl παρέχει τη δομή ContinuousSpace, όπου μπορούμε να ορίσουμε μια απόσταση *neighbor search radius*. Για παράδειγμα, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `nearby_ids(model, agent, radius)` η οποία επιστρέφει τα IDs πρακτόρων εντός ακτίνας `radius` γύρω από τον `agent` σε συνεχή χώρο. Αυτό λειτουργεί χάρη σε έναν χωρικό κατακερματισμό που υλοποιεί εσωτερικά (η ContinuousSpace χωρίζεται σε κυψέλες μεγέθους *spacing* ώστε η αναζήτηση να επιταχύνεται) ⁵⁵. Έτσι, αντί για χειροκίνητη υλοποίηση kd-tree, μπορούμε να αξιοποιήσουμε αυτή την έτοιμη λειτουργία.
- Για κάθε ζεύγος agent-γείτονα, μπορούμε να υπολογίσουμε μια απωθητική δύναμη. Π.χ. μια τυπική επιλογή (όπως στο Helbing 1995) είναι $F_{\text{rep}} = A \exp((r_{ij} - d_{ij})/B) \mathbf{n}_{ij}$, όπου d η μοναδιαία από i προς j . Παρόμοια, για κάθε agent και κάθε τοίχο (ή εμπόδιο), μπορούμε να υπολογίσουμε μια δύναμη όταν ο agent πλησιάζει κάτω από κάποια απόσταση. r_{ij} η απόσταση κέντρων, r_{ij} το άθροισμα των ακτινών (αντιπροσωπεύει το όριο μη σύγκρουσης), και $\mathbf{n}_{ij}$
- Αν ορίσουμε κατάλληλα τις παράμετρους (π.χ. μεγέθη δυνάμεων), μπορούμε να προσεγγίσουμε την ίδια συμπεριφορά αποφυγής συγκρούσεων που έχει το Cromosim ². Η **επιθυμητή κατεύθυνση** του agent θα δίνεται από τη διαφορά θέσης του από τον επόμενο στόχο (π.χ. έξοδο), ομαλοποιημένη στο μέτρο της επιθυμητής ταχύτητας (ή μέγιστης ταχύτητας του). Έτσι, μπορούμε να προσθέσουμε και μια **ελκτική “δύναμη στόχου”** $F_{\text{goal}} = m (v_0 \mathbf{e}_{\text{goal}} - v) / \tau$, που ωθεί την ταχύτητα του agent να τείνει προς v_0 στην κατεύθυνση του στόχου με κάποιο χρόνο αντίδρασης τ . Όλες αυτές οι δυνάμεις αθροίζονται και δίνουν μια επιτάχυνση, την οποία εφαρμόζουμε ενημερώνοντας την ταχύτητα και θέση του agent.}
- Το αποτέλεσμα θα είναι ένα **Social-Force-like μοντέλο** στο Agents.jl. Η Julia μάλιστα επιτρέπει εύκολα το vectorization και την χρήση StaticArrays για γρήγορες πράξεις σε διανύσματα, οπότε ο υπολογισμός δυνάμεων μπορεί να γίνει αποδοτικά. Με αυτό τον τρόπο, **πράκτορες που κινούνται σε συνεχή χώρο με Agents.jl μπορούν να αποκτήσουν ρεαλιστική αποφυγή μεταξύ τους και από τους τοίχους**, όπως στο Cromosim.
- **Μοντέλο Granular (Προβολή ταχυτήτων για μη-επικάλυψη):** Αυτή η προσέγγιση είναι πιο περίπλοκη, αλλά μπορεί επίσης να αναπαραχθεί:

- Στο granular μοντέλο του Cromosim, η ιδέα είναι να λύσουμε ένα πρόβλημα βελτιστοποίησης: βρες τις νέες θέσεις (ή ταχύτητες) των πρακτόρων που ελαχιστοποιούν την απόκλιση από τις επιθυμητές θέσεις (ή ταχύτητες) αλλά δεν έχουν επικαλύψεις ⁵. Αυτό μπορεί να διατυπωθεί ως πρόβλημα **quadratic programming** με περιορισμούς (ή ως προβολή σε σύνολο εφικτών).
- Στο οικοσύστημα Julia υπάρχει το **JuMP.jl** (για μοντελοποίηση βελτιστοποίησης) και solvers όπως το **OSQP** ή **CPLEX/Gurobi** που μπορούν να λύσουν τέτοια προβλήματα σχετικά γρήγορα. Θα μπορούσαμε ανά χρονικό βήμα να θέτουμε: για κάθε agent i μεταβλητές τις νέες συντεταγμένες x_i, y_i , ελαχιστοποίηση $\sum_i |(x_i, y_i) - [(x_i, y_i)_{\text{desired}}]|^2$ με περιορισμούς $|(x_i - x_j, y_i - y_j)| \geq r_i + r_j$ για κάθε ζεύγος i, j (και παρόμοιοι περιορισμοί για απόσταση από τοίχους). Λύνοντας αυτό το QP, θα παίρναμε νέες θέσεις χωρίς συγκρούσεις.
- Αυτή η λύση φυσικά είναι υπολογιστικά ακριβή για μεγάλο πλήθος (το Cromosim χρησιμοποιεί ειδικευμένες μεθόδους – στην υλοποίησή του πιθανόν έναν custom solver με CVXOPT ⁴⁰). Σε Julia, η χρήση ενός έτοιμου QP solver σε κάθε βήμα για πολλούς agents θα ήταν μάλλον αργή. Εναλλακτικά, μπορεί να γίνει μια προσεγγιστική υλοποίηση: π.χ. ένας απλός αλγόριθμος **“υποχώρησης” (iterative relaxation)** όπου αν δύο agents επικαλύπτονται, τους μετακινούμε ελαφρώς αντίθετα ώπου να μην ακουμπάνε. Τέτοιοι αλγόριθμοι μπορούν να γραφούν εύκολα και αν ο αριθμός συγκρούσεων κάθε βήμα είναι μικρός συγκριτικά, θα συγκλίνουν γρήγορα.
- Συμπερασματικά, ενώ το ακριβές granular μοντέλο είναι απαιτητικό, **είναι δυνατόν να προσεγγιστεί στο Agents.jl** είτε με ενσωμάτωση εξωτερικού solver είτε με custom iterative αλγόριθμο. Πολλές φορές, όμως, ένα καλούδιασμένο Social Force μοντέλο αρκεί – το granular ενδείκνυται κυρίως για καταστάσεις εξαιρετικά υψηλής πυκνότητας όπου οι απωθητικές δυνάμεις μπορεί να οδηγήσουν σε αριθμητικές ταλαντώσεις. Η επιλογή εξαρτάται από τις ανάγκες του μοντέλου σας.
- **Ορισμός περιβάλλοντος (Domain) και υπολογισμός πεδίων:** Το Cromosim αυτοματοποιεί τη δημιουργία πεδίου αποστάσεων προς εξόδους κλπ. Στη Julia υπάρχουν τρόποι να γίνει κάτι αντίστοιχο:
- Εφόσον ήδη στο μοντέλο σας υπάρχει έννοια **“χάρτη συγκέντρωσης”** (πιθανότατα ένα πλέγμα cells που καλύπτει τον χώρο), θα μπορούσαμε να χρησιμοποιήσουμε το ίδιο πλέγμα για να υπολογίσουμε και ένα πεδίο απόστασης προς τον στόχο. Για παράδειγμα, αν έχουμε έναν πίνακα όπου τα εμπόδια σημειώνονται (π.χ. cells με τιμή 1 για τοίχο, 0 για ελεύθερο), μπορούμε να εκτελέσουμε έναν **αλγόριθμο Breadth-First Search (BFS)** ή **Dijkstra** από το κελί-στόχο ώστε να βρούμε την απόσταση κάθε κελιού από τον στόχο (σε αριθμό βημάτων ή σε μέτρα αν κάθε κελί έχει γνωστό μέγεθος). Υπάρχουν υλοποιήσεις σε καθαρή Julia για BFS/Dijkstra (π.χ. στο πακέτο Graphs.jl μπορούμε να μετατρέψουμε το grid σε γράφο κόμβων). Ακόμα, για ευκλείδειες αποστάσεις σε συνεχή χώρο με εμπόδια, υπάρχει η μέθοδος **Fast Marching** – ίσως υπάρχει κάποια υλοποίηση Julia (αν όχι, θα μπορούσε να κληθεί μέσω PyCall η αντίστοιχη της SciPy). Επίσης, από πλευράς επεξεργασίας εικόνων, στη Julia υπάρχει το **Images.jl** που πιθανώς περιέχει συναρτήσεις για *distance transform* σε binary εικόνες (δηλ. για κάθε pixel μη-τοίχο να βρίσκει την απόσταση από τον πλησιέστερο τοίχο).
- Αφού έχουμε έναν πίνακα αποστάσεων προς την έξοδο, μπορούμε να δημιουργήσουμε και έναν πίνακα κατεύθυνσης: για κάθε κελί, υπολογίζουμε το τοπικό **gradient** του πεδίου αποστάσεων (π.χ. διαφορές με τους γείτονες) και αυτό δίνει ένα διάνυσμα κατεύθυνσης προς το κατήφορο (δηλ. προς την έξοδο). Μπορούμε να αποθηκεύσουμε αυτά τα διανύσματα σε έναν πίνακα ή, πιο απλά, να τα υπολογίζουμε επί τόπου όταν ένας agent χρειάζεται καθοδήγηση.
- Στο Agents.jl, ο χώρος μπορεί να παραμείνει συνεχής. Δηλαδή οι πράκτορες έχουν συνεχή θέση (x, y) , αλλά όταν χρειάζεται να βρουν κατεύθυνση, κοιτάμε σε ποιο κοντινό κελί του πλέγματος βρίσκονται και παίρνουμε το διάνυσμα από εκεί. Αυτό είναι ακριβώς που κάνει

εσωτερικά και το Cromosim (pixel grid + συνεχής κίνηση πάνω του). Έτσι, **μπορούμε να αναπαράγουμε την πλοήγηση τύπου Cromosim**.

- Αν δεν θέλουμε να διαχειριστούμε πλέγματα, μια άλλη επιλογή στη Julia είναι η χρήση ενός πακέτου pathfinding όπως το **Pathfinding.jl** ή κάποια υλοποίηση A* που μπορεί να λειτουργεί σε συνεχή χώρο με πολυγωνικά εμπόδια. Υπάρχουν αλγόριθμοι υποδιαίρεσης χώρου σε τρίγωνα (αφού καθοριστούν οι πολύγωνα περιοχές ελεύθερου χώρου) για εύρεση διαδρομής. Ωστόσο, αυτό ίσως είναι υπερβολή εάν ένα απλό grid αρκεί.
- Τέλος, επειδή το Agents.jl μπορεί να λειτουργήσει και σε **GridSpace** (διακριτό χώρο), θα υπήρχε η επιλογή να μεταφέρουμε όλη την προσομοίωση σε ένα grid (π.χ. cell occupancy, όπως στα cellular automata). Όμως αυτό θα άλλαζε τη φύση του μοντέλου (π.χ. δεν θα υπάρχει πια συνεχής ταχύτητα αλλά βήματα από κελί σε κελί). Για να διατηρήσουμε τη ρεαλιστικότητα, προτείνεται ο συνδυασμός: πλέγμα για **υπολογισμούς πεδίου** (navigation, συγκέντρωση τοξικού) και **συνεχής θέσεις** για τους agents.
- **Πολλαπλοί προορισμοί και όροφοι:** Στο Cromosim αυτό γίνεται με πολλαπλά Domain και Destination objects. Στο Agents.jl, δεν υπάρχει έτοιμη έννοια "Domain", αλλά μπορούμε απλά να προσθέσουμε ένα πεδίο `floor` σε κάθε agent και να τονίζουμε το περιβάλλον:
- Μπορούμε να διατηρούμε ξεχωριστούς πίνακες αποστάσεων για κάθε έξοδο ή επόμενη στάση. Π.χ. ένας agent μπορεί να έχει μια λίστα προορισμών `destinations = [(x1,y1), (x2,y2), ...]`. Στο βήμα προσομοίωσης ελέγχουμε: αν ο agent έφτασε κοντά στο $(x1,y1)$, τότε αφαιρούμε το πρώτο και χρησιμοποιούμε το επόμενο ως νέο στόχο. Αυτό δίνει μια αλυσίδα στόχων.
- Για σενάριο πολλών ορόφων, μια απλή πρακτική είναι να **τρέχει η προσομοίωση όλους τους agents μαζί** αλλά να έχουμε ένα πεδίο `agent.floor`. Τα εμπόδια/τοξικότητα κλπ. θα μπορούσαν να κωδικοποιηθούν σε 3D πίνακες [όροφος, x, y] ή, απλούστερα, να έχουμε έναν ξεχωριστό πίνακα πεδίου ανά όροφο. Όταν ένας agent αλλάζει όροφο (π.χ. φτάνει σε θέση σκάλας), απλώς αλλάζουμε το `agent.floor` και ενημερώνουμε τη θέση του στη νέα συντεταγμένη (ίσως και αλλάζουμε το ContinuousSpace αν έχουμε ορίσει διαφορετικό χώρο ανά όροφο – αλλά πιθανόν μπορούμε να αναπαραστήσουμε ένα κτίριο και ως 3D continuous space όπου το z είναι επίπεδο).
- Αν η τρίτη διάσταση (υψος) δεν παίζει ρόλο στην κίνηση πέρα από διαχωρισμό ορόφων, μπορούμε να αποφύγουμε πραγματικό 3D simulation. Αλλά αξίζει να σημειωθεί ότι το Agents.jl **μπορεί να λειτουργήσει σε 3D continuous space** εγγενώς, οπότε θα μπορούσαμε να δώσουμε σε κάθε agent συντεταγμένες (x, y, z) όπου το z είναι 0 για ισόγειο, 3 για 1ος όροφος, κλπ., και οι σκάλες θα ήταν γραμμές που αλλάζουν το z του agent όταν φτάσει εκεί. Είναι λίγο πιο περίπλοκο στον χειρισμό (και στην οπτικοποίηση), γι' αυτό ίσως προτιμότερο να αντιμετωπιστεί λογικά με πεδίο `floor`.
- Συμπερασματικά, **η λειτουργία πολλαπλών διαδοχικών προορισμών του Cromosim μπορεί να αναπαραχθεί** με απλές δομές δεδομένων στο Agents.jl. Δεν υπάρχει τεχνικός περιορισμός – είναι θέμα λογικής στο update function των agents.
- **Καταγραφή διελεύσεων (αισθητήρες):** Αν θέλουμε να έχουμε ανάλογα αποτελέσματα με τους αισθητήρες του Cromosim:
- Ορίζουμε γραμμές/περιοχές ως «εξόδους» ή checkpoints. Για κάθε τέτοια γραμμή, μπορούμε σε κάθε βήμα να ελέγχουμε όλους τους agents: αν ένας agent βρίσκεται πριν από τη γραμμή στο προηγούμενο βήμα και μετά τη γραμμή στο επόμενο, σημαίνει ότι διέσχισε τη γραμμή κατά το

διάστημα αυτό. Η μαθηματική ανίχνευση μπορεί να γίνει με έλεγχο διαφοράς πρόσημου ως προς την εξίσωση της γραμμής.

- Εναλλακτικά, μπορούμε να το απλοποιήσουμε: ορίζουμε μια μικρή ζώνη (π.χ. ορθογώνιο πλάτους ελάχιστου γύρω από τη γραμμή) και απλά καταγράφουμε τότε ένας agent μπαίνει σε αυτήν τη ζώνη (που αντιστοιχεί ακριβώς στο πέρασμα της γραμμής). Στο Agents.jl, επειδή έχουμε πλήρη πρόσβαση στις θέσεις, είναι εύκολο να γράψουμε αυτή τη συνθήκη.
- Η εξαγωγή των δεδομένων μπορεί να γίνει με τον μηχανισμό data collection: π.χ. να συλλέγουμε ένα vector με timestamps διελεύσεων ή να αυξάνουμε έναν μετρητή. Στο τέλος μπορούμε να χρησιμοποιήσουμε το Plotting (PyPlot.jl ή Makie.jl) για να φτιάξουμε τα ίδια γραφήματα ροής που βγάζει και το Cromosim ²⁸.
- Επίσης, αν θέλουμε on-line υπολογισμό, μπορούμε σε κάθε βήμα να υπολογίζουμε τη στιγμιαία ροή (πλήθος διελεύσεων στο τελευταίο Δt) και να την αποθηκεύουμε. Το Agents.jl επιτρέπει συλλογή δεδομένων ανα βήμα χωρίς να μπερδεύει τον κώδικα του update (μέσω των `step!(model, agent_step!, model_step!, n_steps)` και `adata`/`mdata` arguments μπορούμε να ορίσουμε τι να αποθηκεύει κάθε φορά).

• **Τυχαιοποίηση παραμέτρων και ομάδες:** Αυτά είναι απλούστατα στη Julia:

- Η δημιουργία ομάδων με διαφορετικές ιδιότητες (ακτίνα, ταχύτητα, προορισμός) μπορεί να γίνει με απλές λούπες κατά την αρχικοποίηση του μοντέλου: π.χ. δημιουργούμε `N` agents, για τους μισούς δίνουμε `speed = rand(Normal(1.5, 0.2))` και `destination = A`, για τους άλλους μισούς `speed = rand(Normal(1.2, 0.2))` και `destination = B`.
- Η Julia έχει ισχυρό random API (Random stdlib) οπότε οποιαδήποτε κατανομή (κανονική, ομοιόμορφη, κ.ά.) μπορεί να δειγματοστεί για να δώσουμε ποικιλία. Αυτό ανταποκρίνεται πλήρως στη δυνατότητα του Cromosim να ορίζει **κατανομές ακτίνας/ταχύτητας** ¹⁶.
- Εάν θέλαμε να ενσωματώσουμε έννοιες όπως *groups* (ομαδοποιημένη συμπεριφορά), π.χ. κάποιοι agents να θεωρούνται παρέα και να μένουν κοντά, μπορούμε να εισάγουμε λογική που αυξάνει μια ελκτική δύναμη μεταξύ πρακτόρων που ανήκουν στην ίδια ομάδα. Τέτοιες επεκτάσεις είναι εύκολες στο Agents.jl διότι απλώς προσθέτουμε μια συνθήκη στον υπολογισμό δυνάμεων.

Συμπερασματικά, **όλες οι βασικές λειτουργίες του Cromosim μπορούν να υλοποιηθούν στο Agents.jl**, χρησιμοποιώντας συνδυασμό των δυνατοτήτων της Julia και πρόσθετων πακέτων αν χρειαστεί. Το κλειδί είναι να αναπαραστήσουμε ρητά ό,τι στο Cromosim είναι αυτοματοποιημένο: τη γεωμετρία (με πίνακες ή εξισώσεις), το μοντέλο κίνησης (με κώδικα κανόνων), και τις μετρικές (με custom καταμέτρηση). Το υπάρχον σύστημα σας έχει ήδη υλοποιήσει κάποια από αυτά (agents σε continuous space, πεδίο τοξικότητας, μεταβολή ταχύτητας). Σε αυτό μπορούμε να “κουμπώσουμε” τα επιπλέον στοιχεία: π.χ. **προσθήκη αποφυγής σύγκρουσης** μεταξύ πρακτόρων, **πλοήγηση** βάσει πεδίου αποστάσεων αντί απλής ευθείας προς τον στόχο, κ.ο.κ.

Είναι σημαντικό να αξιοποιηθούν και οι **βιβλιοθήκες Julia**: - Για υπολογισμούς πεδίων αποστάσεων/διαδρομών: π.χ. `ImageDistances.jl` ή `Dijkstra` από `Graphs.jl` όπως αναφέρθηκε. - Για γρήγορη εύρεση γειτόνων: η ίδια η `ContinuousSpace` το κάνει, αλλά αν χρειαζόμαστε πιο προσαρμοσμένο έλεγχο, υπάρχει και το `NearestNeighbors.jl` που υλοποιεί kd-tree και ball-tree αναζητήσεις σε σημεία. - Για επίλυση βελτιστοποίησης: `JuMP.jl` + solver, αν θέλουμε ακριβές granular. - Για απεικόνιση: το **Makie.jl** μπορεί να παράγει πλούσια γραφικά. Μπορούμε να φτιάξουμε ένα scatter plot των agents ανά βήμα, να σχεδιάσουμε γραμμές για τους τοίχους, και ακόμη και να δούμε 3D (π.χ. αν έχουμε όροφο μπορεί να δούμε μια 3D άποψη του κτιρίου με σφαίρες ως agents). Υπάρχει και το **InteractiveDynamics.jl** που παρέχει κάποιες έτοιμες συνιστώσες για interactive visualization των μοντέλων του Agents.jl.

Τελικά, η **επιδίωξη αναπαραγωγής του Cromosim στο Agents.jl** δεν σημαίνει απαραίτητα γραμμή-προς-γραμμή αντιγραφή αλγορίθμων, αλλά μάλλον αξιοποίηση των ίδιων αρχών: - χρήση πεδίων δυναμικού για καθοδήγηση πλήθους, - χρήση δυνάμεων ή/και περιορισμών για αποφυγή συγκρούσεων, - παρακολούθηση κρίσιμων μεγεθών (χρόνοι διέλευσης, ροές), - τυχαιοποίηση και ετερογένεια στους agents.

Το Agents.jl δίνει το πλαίσιο να υλοποιήσετε αυτά τα στοιχεία **με το δικό σας ρυθμό και έλεγχο**. Μπορείτε να ξεκινήσετε απλά (π.χ. προσθέτοντας μόνο μια βασική απωθητική δύναμη μεταξύ κοντινών agents) και να επεκτείνετε σταδιακά (π.χ. πιο σύνθετο μοντέλο δύναμης, ή προσθήκη pathfinding).

Δυνατότητες και απαιτήσεις εκτός τυπικού πλαισίου

Στην πορεία της υλοποίησης, αξίζει να αναφερθούμε σε ορισμένες **δυνατότητες που ούτε το Cromosim ούτε το Agents.jl καλύπτουν απόλυτα out-of-the-box**, και οι οποίες ενδεχομένως απαιτούν σημαντική επιπλέον προσπάθεια ή εξωτερικά εργαλεία:

- **Τρισδιάστατη προσομοίωση και οπτικοποίηση (3D):** Το Cromosim, όπως είδαμε, είναι αυστηρά δισδιάστατο ως προς την κίνηση και την απεικόνιση ⁴⁶. Το Agents.jl μπορεί να χειριστεί 3D συντεταγμένες, αλλά η **οπτικοποίηση 3D πλήθους** είναι ένα ξεχωριστό ζήτημα. Αν ο στόχος ήταν, για παράδειγμα, να δημιουργηθεί ένα ρεαλιστικό 3D περιβάλλον (με τρισδιάστατα μοντέλα ανθρωπάκια, κτίρια κλπ.) και οι πράκτορες να κινούνται μέσα σε αυτό, τότε θα χρειαζόταν ενδεχομένως η διασύνδεση με μια game engine ή μια βιβλιοθήκη γραφικών. Στο οικοσύστημα Julia δεν υπάρχει (μέχρι στιγμής) κάποιο έτοιμο εργαλείο τύπου “Crowd3D”, οπότε θα έπρεπε να εξεταστεί η χρήση π.χ. του **Unity** ή **Unreal Engine** σε συνδυασμό με το simulation (π.χ. τρέχοντας την προσομοίωση στη Julia και στέλνοντας τις θέσεις σε μια μηχανή για απεικόνιση). Αυτή η προσέγγιση απαιτεί σημαντική προσπάθεια και έξτρα κώδικα (APIs, συγχρονισμός κλπ.). Μια άλλη επιλογή θα ήταν η χρήση του **Makie.jl** για 3D rendering: το Makie μπορεί να σχεδιάσει σφαίρες και κυλίνδρους σε 3D σκηνή, οπότε θα μπορούσε να δώσει μια απλοποιημένη 3D απεικόνιση (όχι ρεαλιστικά μοντέλα ανθρώπων, αλλά τουλάχιστον ένα τρισδιάστατο διάγραμμα). Και πάλι, δεν είναι “έτοιμο κουτί” – θα πρέπει ο προγραμματιστής να το φτιάξει. Εν κατακλείδι, **η 3D απεικόνιση είναι μια δυνατότητα που δεν υλοποιείται εύκολα ούτε στο Cromosim ούτε στο Agents.jl** χωρίς εξωτερικά εργαλεία.

- **Διαδραστικό περιβάλλον προσομοίωσης (real-time interface):** Κάποια εμπορικά λογισμικά προσομοίωσης (π.χ. Pathfinder, MassMotion) έχουν interactive GUI όπου ο χρήστης μπορεί να τοποθετεί πράκτορες, να ξεκινά/σταματά την προσομοίωση, να αλλάζει παραμέτρους εν κινήσει. Το Cromosim και το Agents.jl είναι περισσότερο **offline simulators**: τρέχεις το σενάριο και μετά βλέπεις τα αποτελέσματα. Αν απαιτείται μια διεπαφή όπου κάποιος χειριστής μπορεί σε πραγματικό χρόνο να μεταβάλλει κάτι (π.χ. να κλείσει μια πόρτα και να δει το αποτέλεσμα στην εκκένωση εκείνη τη στιγμή), τότε θα πρέπει να υλοποιηθεί ένα custom interface. Στη Julia, μια πιθανότητα είναι να χρησιμοποιηθεί το **Gtk.jl** (GUI library) ή ένα web-based UI (π.χ. **Dash.jl** ή **Genie.jl** για web dashboards) που να συνδέεται με το Agents.jl model. Για παράδειγμα, θα μπορούσε να υπάρχει ένα κουμπί “κλείσε έξοδο Α” που όταν πατηθεί να ενημερώνει το περιβάλλον (π.χ. να θέτει κάποια cells ως εμπόδια) και οι πράκτορες αμέσως να αναπροσαρμόζουν την πορεία τους. Αυτά όμως απαιτούν **προγραμματισμό επιπλέον** και δεν υπάρχουν έτοιμα. Το θετικό είναι ότι το Agents.jl προσφέρει hooks για interactive use (μπορείς να “βλέπεις” μέσα στο μοντέλο ενώ τρέχει σε REPL ή notebook). Συνοψίζοντας, ένα πραγματικό **interactive simulation tool** θα απαιτούσε σημαντικό χρόνο ανάπτυξης, και αυτό είναι πέρα

από το scope τόσο του Cromosim (που δεν το επιχειρεί καν) όσο και του Agents.jl (που δίνει όμως τα building blocks αν κάποιος θέλει να το φτιάξει).

- **Μοντέλα προχωρημένης συμπεριφοράς:** Αν το project απαιτούσε πέρα από την φυσική κίνηση και **περίπλοκη συμπεριφορά πρακτόρων** (π.χ. μοντέλα αποφάσεων, αντίληψης, πανικού, συνεργασίας), ούτε το Cromosim έχει κάτι τέτοιο έτοιμο (είναι αρκετά “φυσικό” μοντέλο, δεν ενσωματώνει ψυχολογικές καταστάσεις), αλλά το Agents.jl θα διευκόλυνε την ενσωμάτωση. Για παράδειγμα, μπορούμε να επεκτείνουμε το Agents.jl model ώστε οι πράκτορες να έχουν κατάσταση “calm” ή “panicked” που επηρεάζει την ταχύτητά τους ή την τάση τους να συνωστίζονται. Αυτό θα απαιτούσε σχεδιασμό κανόνων, αλλά είναι εφικτό. Σε Cromosim, θα έπρεπε να φτιάξουμε ξεχωριστό model από την αρχή. Άρα, αν ποτέ ζητηθούν τέτοιες δυνατότητες, η πλατφόρμα Julia/Agents.jl είναι πιο βιώσιμη.
- **Επικύρωση και πιστοποίηση:** Ένα πλεονέκτημα του Cromosim είναι ότι υλοποιεί μοντέλα από βιβλιογραφία που είναι θεωρητικά επικυρωμένα (π.χ. το social force). Το Agents.jl model που θα δημιουργήσουμε θα χρειαστεί **επαλήθευση** ότι συμπεριφέρεται ρεαλιστικά (δεν συγκρούονται οι πράκτορες, οι χρόνοι εκκένωσης βγάζουν νόημα κτλ.). Η διαδικασία αυτή είναι ουσιαστικά η ίδια ανεξαρτήτως πλατφόρμας: θα συγκρίνουμε αποτελέσματα με γνωστές καταστάσεις ή με το Cromosim ως σημείο αναφοράς. Για παράδειγμα, θα μπορούσαμε να τρέξουμε το ίδιο σενάριο εκκένωσης σε Cromosim και Agents.jl και να συγκρίνουμε τους χρόνους εξόδου ή τη ροή στην πόρτα. Αυτό θα δείξει αν η αναπαραγωγή μας είναι **ακριβής** ή αν χρειάζεται ρύθμιση παραμέτρων.
- **Εξωτερικές ολοκληρώσεις:** Αν απαιτηθεί σύνδεση με άλλες εφαρμογές – π.χ. ένα CAD πρόγραμμα για να διαβάσουμε απευθείας κατόψεις κτιρίων, ή μια εφαρμογή GIS για δεδομένα πληθυσμού – αυτά ούτε το Cromosim διαθέτει έτοιμα. Στη Julia όμως μπορούμε να αξιοποιήσουμε πακέτα όπως το **Shapefile.jl** (ανάγνωση shapefiles), **ArchGDAL.jl** (για GIS raster/vector) κ.ά., για να διαβάσουμε τέτοιες πληροφορίες και να τις ταιύσουμε στο Agents.jl model. Αυτή είναι μια δύναμη της Julia: η ενσωμάτωση διαφόρων formats και εργαλείων, ενώ το Cromosim βασικά δέχεται μόνο εικόνα/JSON. Βέβαια, και πάλι, αυτό απαιτεί από τον προγραμματιστή να κάνει την κόλλα μεταξύ των εργαλείων.
- **Παράλληλη εκτέλεση πολλαπλών σεναρίων ή πολύ μεγάλων πλήθων:** Όπως αναφέρθηκε, το Agents.jl υποστηρίζει πιο εύκολα την παράλληλη επεξεργασία. Αν, για παράδειγμα, θέλατε να δοκιμάσετε **πολλά σενάρια** (διαφορετικές παραμετροποιήσεις) για να κάνετε ανάλυση ευαισθησίας, το Agents.jl έχει ενσωματωμένο μηχανισμό batch run (μπορεί να τρέξει π.χ. 100 simulations με διαφορετικές τιμές σε παράλληλα threads και να συλλέξει αποτελέσματα). Αυτό το **είδος πειραματισμού** θα ήταν πολύ χρήσιμο αν θέλετε να δοκιμάσετε π.χ. διάφορες στρατηγικές εκκένωσης υπό τοξικές συνθήκες. Στο Cromosim θα έπρεπε να γραφτεί ένα ξεχωριστό Python script να κάνει loop στα σενάρια – εφικτό αλλά όχι τόσο out-of-the-box.

Τελειώνοντας, μπορούμε να πούμε ότι η μετάβαση ή **ενοποίηση του μοντέλου σας στο Agents.jl με χαρακτηριστικά τύπου Cromosim είναι όχι μόνο εφικτή αλλά και ωφέλιμη** μακροπρόθεσμα. Θα έχετε ένα **ενιαίο σύστημα** μέσα στη Julia που προσομοιώνει και την κίνηση του πλήθους και την εξάπλωση/επίδραση του τοξικού παράγοντα. Αυτό το σύστημα θα είναι ευέλικτο για μελλοντικές επεκτάσεις (π.χ. νέα είδη πρακτόρων, νέα φαινόμενα), εκμεταλλευόμενο τις βιβλιοθήκες υψηλής απόδοσης της Julia. Η κύρια πρόκληση είναι η **αρχική υλοποίηση** ορισμένων λειτουργιών που το Cromosim “χαρίζει” έτοιμες – ωστόσο, όπως καθοδηγήθηκε παραπάνω, υπάρχουν σαφή μονοπάτια υλοποίησης καθενός. Με προσεκτική υλοποίηση, μπορείτε να φτάσετε σε ένα αποτέλεσμα όπου το Agents.jl μοντέλο **μιμείται πιστά το Cromosim** ως προς την συμπεριφορά πλήθους, ενώ παράλληλα συνεχίζει να υποστηρίζει τις μοναδικές σας ανάγκες (τοξικολογία, κ.λπ.).

Βιβλιογραφία / Πηγές: Το παρόν κείμενο συνδυάζει πληροφορίες από την τεκμηρίωση του Cromosim ⁵⁶ ³, αξιολογήσεις λογισμικού ⁵², καθώς και την επίσημη τεκμηρίωση/άρθρα του Agents.jl ⁵⁴. Οι συγκριτικές παρατηρήσεις βασίζονται σε αυτά τα δεδομένα και σε γνώση των δυνατοτήτων των δύο περιβαλλόντων όπως εκτίθενται στις πηγές.

¹ ⁶ ²⁸ ⁴⁶ ⁴⁷ ⁴⁸ ⁵¹ ⁵² ⁵³ urban-analytics.github.io

https://urban-analytics.github.io/dust/docs/ped_sim_review.pdf

² ³ ⁴ ⁵ ³³ ³⁴ ⁴⁵ [Make a microscopic simulation — cromosim documentation](https://www.cromosim.fr/1.0/example_micro.html)

https://www.cromosim.fr/1.0/example_micro.html

⁷ ⁸ ¹³ ¹⁴ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ²⁶ ²⁷ ³⁷ ³⁸ ³⁹ ⁴⁰ ⁴² ⁴³ ⁴⁴ [Microscopic models : granular and social force model — cromosim documentation](https://www.cromosim.fr/micro.html)

<https://www.cromosim.fr/micro.html>

⁹ ¹⁰ ¹¹ ¹² ¹⁵ ⁵⁶ [Build a domain — cromosim documentation](https://www.cromosim.fr/example_dom.html)

https://www.cromosim.fr/example_dom.html

¹⁶ ¹⁷ ³⁵ ³⁶ ⁴¹ [Welcome to cromosim's documentation! — cromosim documentation](https://www.cromosim.fr)

<https://www.cromosim.fr>

²⁹ ³⁰ ³¹ ³² [GitHub - sylvain-faure/cromosim: Crowd Motion Simulation](https://github.com/sylvain-faure/cromosim)

<https://github.com/sylvain-faure/cromosim>

⁴⁹ ⁵⁰ ⁵⁴ [Reliable and Efficient Agent-Based Modeling and Simulation](https://www.jasss.org/27/2/4.html)

<https://www.jasss.org/27/2/4.html>

⁵⁵ [Tutorial · Agents.jl](https://juliadynamics.github.io/Agents.jl/v4.0/tutorial/)

<https://juliadynamics.github.io/Agents.jl/v4.0/tutorial/>